JOHANNES KEPLER
UNIVERSITY LINZ

Simon Grundner
Institute for Signal
Processing

@ k12136610@students.jku.at
https://jku.at/isp

October 5, 2025

04 - Delta ADC

Hardwaredesign PR - Exercise 04

Semester 2024W

ISP
INSTITUTE OF SIGNAL PROCESSING

Keywords Delta ADC, PWM, Strobe Signals

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

1. Delta ADC

Delta ADC (Nachlaufumsetzer)

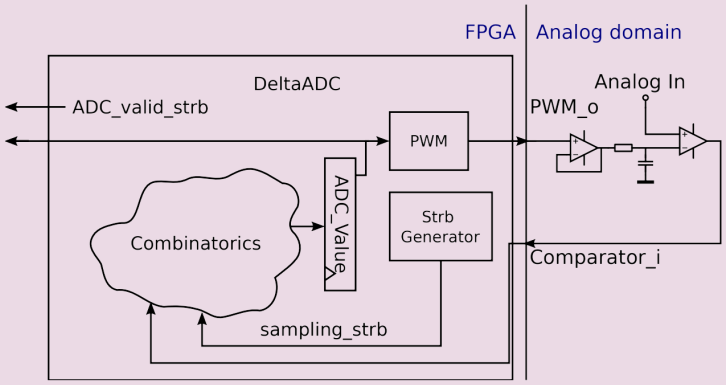


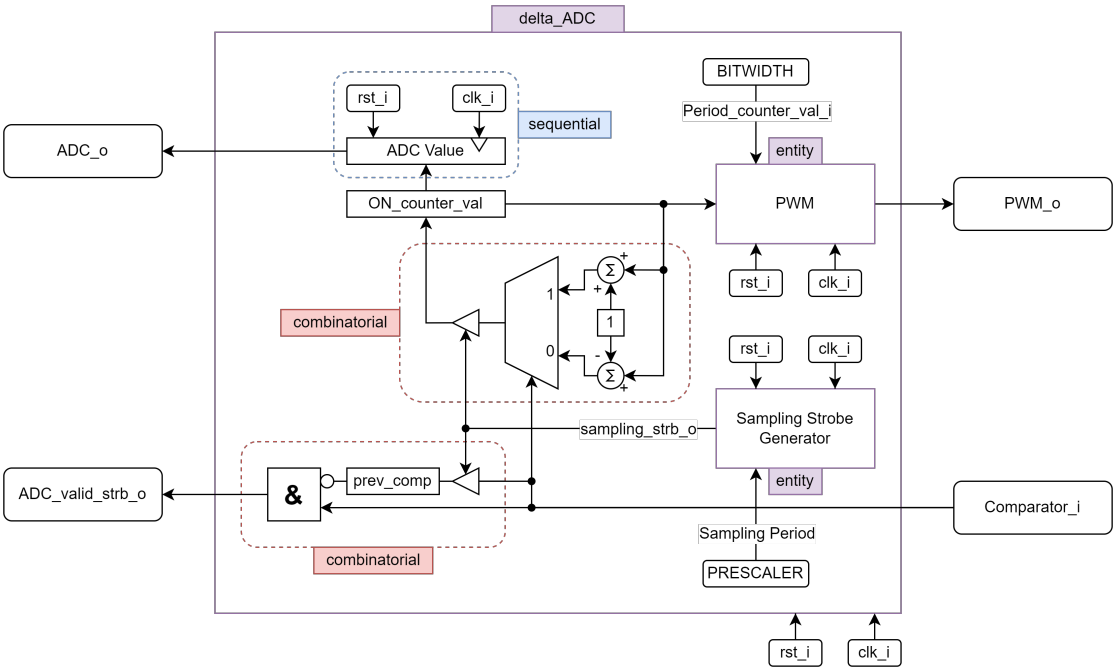
Figure 1: Delta ADC Block

In this exercise, we will start the digital design of a delta ADC, schematically shown in Figure 1. It works like the following. A PWM Signal generated by the FPGA is output through an analog low pass filter (more details on the analog part will follow in a future exercise), resulting in a voltage level that is compared to the analog input via an analog comparator. This comparator outputs a zero if the analog input is lower than the comparison voltage level and a one otherwise. In the digital domain the comparator signal (Comparator i) is used to increase the ADC value in case Comparator i is one and decrease the ADC value in case Comparator i is zero. A change of the ADC value should only occur at rising clock edges where the sampling strobe (sampling strb) generated by a strobe generator is one. This will lead to a sampling that is equidistantly spaced in time with the sampling frequency defined by the frequency of sampling strb. The ADC value is feed to the PWM to define the ON counter val i. This way a simple control loop is closed, allowing the ADC value to adapt to analog signal. This way the ADC signal will follow the analog signal with its value proportional (up to an adjustment error) to the analog input.

To-Do-List

- Draw a block diagram of the required combinatorics. Do you need to have a finite state machine? If you do, please write a state diagram.
- Draw the required blocks to the block diagram for implementing the signal ADC valid strb.
- Write the entities and architectures that are required in VHDL. As you did not design the strobe generator yet it is suggested that you start with the strobe generator. Find meaningful generics such that the frequency of the strobe generator is adjustable (you might set the time between strobes to a few clock cycles for the simulation, however for future use several thousands of clock cycles might be required). Use one .vhd file for each entity and one .vhd file for each architecture.
- Write a testbench in VHDL (extra file). Simulate the system using Modelsim and hand in screenshots showing the output of the waveform window. Find meaningful test cases. As you are not able to simulate the analog domain, your testbench can only simulate the behavior of the comparator input. What are meaningful testcases for Comparator i to adequately simulate the designs behavior.
- Discuss the results of the simulation in a few sentences and answer the questions.

1.1 ADC Module Implementation



Entity

```
entity delta_adc is
    generic (
        BITWIDTH : natural := 8;
        SAMPLING_PSC : natural := 4
    );
    port (
        clk_i               : in std_ulogic;
        rst_i               : in std_ulogic;
        PWM_o               : out std_logic;
        ADC_valid_strobe_o  : out std_ulogic;
        ADC_Value_o         : out unsigned(BITWIDTH-1 downto 0);
        Comparator_i        : in std_logic
    );
end delta_adc;
```

Architecture

The ADC uses a PWM Module and a Sampling-Strobe-Generator. Other than that it is described by three processes.

Signals

```
    constant MAX_COUNTER_VAL : unsigned(BITWIDTH-1 downto 0) := (others => '1');
    signal ON_counter_val    : unsigned(BITWIDTH-1 downto 0) := (others => '0');
    signal sampling_strobe   : std_ulogic := '0';
    signal prev_comp         : std_logic := '0';
```

(1) `adc_reg : process (clk_i, rst_i)`

The register process determines the clock and reset behavior. On the rising edge the next combinatorially determined ADC state is assigned to the ADC output, which is the duty cycle counter value `ON_counter_val` of the PWM generator. On reset, the ADC output is set to `'0'`.

(2) `adc_comb : process (sampling_strobe, Comparator_i)`

The combinatorial logic necessary for determining the ADC value is described here. Assuming that the `sampling_strobe` is `'1'`, the following applies: If the input value `Comparator_i` delivered by the comparator is `'1'`, the duty cycle is incremented by 1, otherwise it is decremented. Additionally, with each `sampling_strobe`, the current comparator state is stored, which is relevant for the validation of the ADC value.

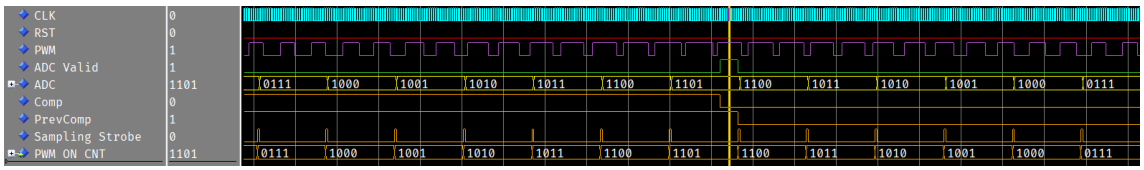
(3) `validate_comb : process (Comparator_i, prev_comp)`

The ADC has an `ADC_valid_strb_o` output, which indicates whether the actual ADC value is present at the output, meaning the settling process is complete. In this implementation, this state occurs if the comparator value reverses. If the comparator value has been `'1'` for several sampling periods and suddenly becomes `'0'` (and vice versa), then the PWM average value in the last sampling period has reached the analog value, and the correct value is approximately present at the ADC output.

Simulation

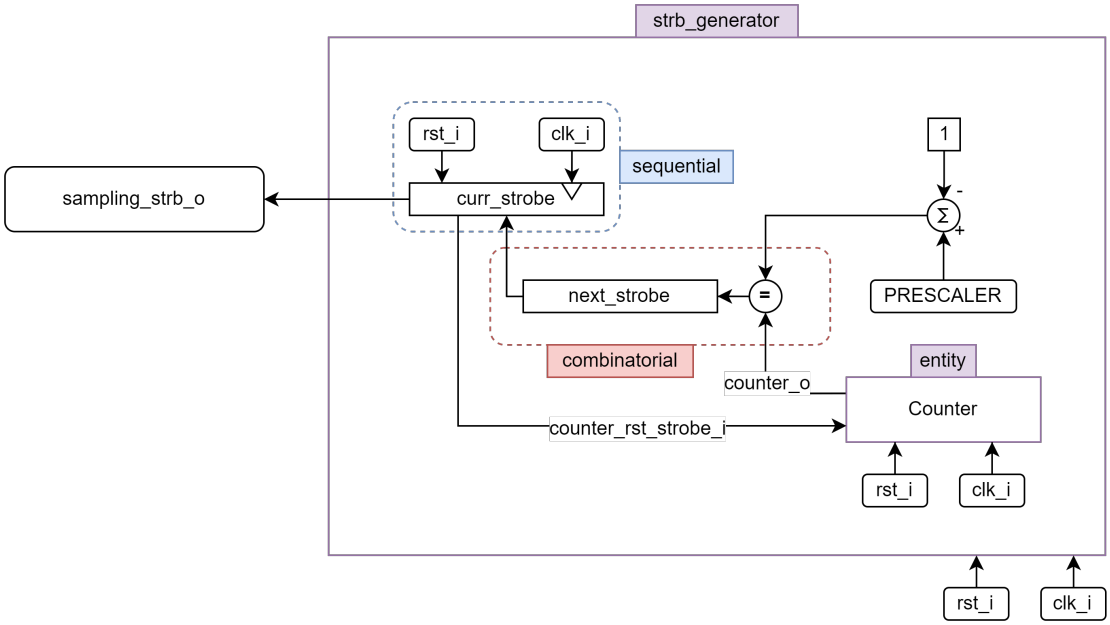
ADC-Test

Here, the comparator input is emulated by setting it to 1 in the stimulus process after a certain time. During this time, you can see how the duty cycle of the PWM signal increases and, as expected, decreases again with the switching of the comparator input. In the sampling period shortly after the switch, it can also be seen how the ADC valid strobe goes high.



In this waveform, the ADC value would be 1101.

1.2 Strobe Generator Module Implementation



Entity

Architecture

Bei dieser Realisierung des Strobe-Generators wurde entschieden, dass die Frequenz, mit der die Strobe Generiert wird keine Zeitdauer, sondern ein Teiler (Prescaler) der Clock ist. Damit kann der sowieso Notwendige Counter auch für die Abzählung der Sampling-Periode verwendet werden.

Signale

```
1 signal next_strobe : std_ulogic := '0';
2 signal curr_strobe : std_ulogic := '0';
3 signal strb_counter : unsigned(PRESCALER-1 downto 0) := (others => '0');
```

(1) `reg_seq : process (clk_i, rst_i)`

Der sequenzielle Prozess bestimmt das Clock- und Resetverhalten. Bei der steigenden Flanke wird der momentanen Strobe `curr_strobe` dem nächsten, kombinatorisch ermittelten Strobe-Zustand `next_strobe` zugewiesen. Beim Reset wird der Strobe-Ausgang auf `'0'` gesetzt.

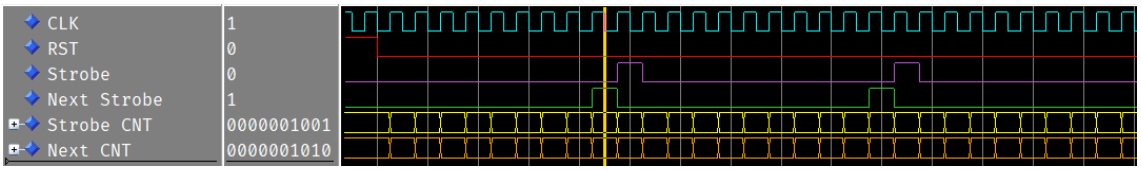
(2) `strb_comb : process (strb_counter)`

In diesem Prozess wird die Kombinatorik zur Bestimmung des nächsten Strobe-Zustands beschrieben. Standardmäßig ist der nächste Strobe Zustand `'next_strobe'` gleich `'0'`. Erreicht der Strobe-Counter jedoch seinen maximalen Wert, wird `next_strobe '1'`. Da die Reset-Strobe des Counters mit `curr_strobe` verbunden ist, wird `next_strobe` zurück auf `'0'` gesetzt. Damit wird erreicht, dass der Strobe-Ausgang genau für eine Periode High ist.

Simulation

Strobe Test

Hier ist das Strobe-Signal mit einem Prescaler von 10 zu sehen, wie es nach 10 Taktflanken für eine Periode lang high wird.



Hier ist die Ermittlung der Counter Länge nicht optimal, da sie durch `unsigned(PRESCALER -1 downto 0)` bestimmt wird. Die größe des Counter Wertes ist daher viel zu groß, da nur bis zum Prescaler Wert gezählt wird und nicht bis 2^{10}. Man müsste den Prescaler entweder als `std_ulogic_vector` übergeben, oder eine art log2 Funktion implementieren.

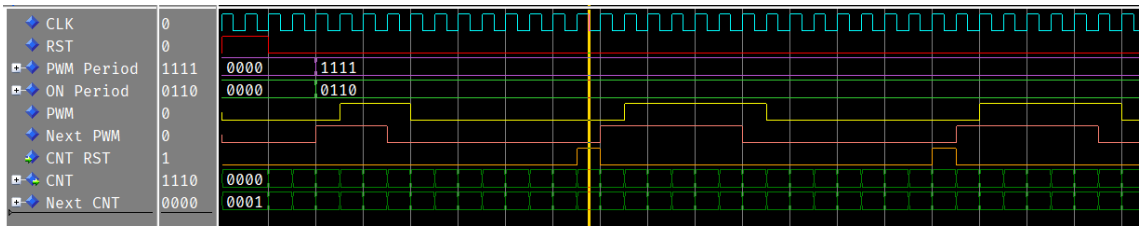
1.3 PWM Module Implementation

PWM Module is implemented as in Exercise 3.

Simulation

PWM-Test 1

In dieser PWM-Testbench wird der PWM-Generator statisch, ohne veränderung des Duty-Cycles betrachtet. Zu sehen ist hier die Konfiguration `Period_counter_val_i = 1111` (15) mit dem Arbeitszyklus
`ON_counter_val_i = 0110` (6). Da der im Zyklus der Wert 0 mit gezählt wird muss bei einer Periode vom 15 also von 0 bis 14 gezählt werden. Nun ist zu sehen, dass der Duty-Cycle von 6 genau sechs Pulse der PWM Periode high ist.



1.4 Counter Module Implementation

The counter is implemented as in the previous exercise 2.